This scope validation check has a performance cost, so by default it's enabled only when your app is running in a development environment, but it should help you catch most problems of this kind. You can enable or disable this check regardless of environment by configuring the `ValidateScopes` option on your `WebApplicationBuilder` in Program.cs by using the `Host` property, as shown in the following listing.

Listing 9.13 Setting the `ValidateScopes` property to always validate scopes

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);  ❶

builder.Host.UseDefaultServiceProvider(o =>                          ❷
{
    o.ValidateScopes = true;                                         ❸
    o.ValidateOnBuild = true;                                        ❹
});
```

❶ The default builder sets ValidateScopes to validate only in development environments.

❷ You can override the validation check with the UseDefaultServiceProvider extension.

❸ Setting this to true will validate scopes in all environments, which has performance implications.

❹ ValidateOnBuild checks that every registered service has all its dependencies registered.

Listing 9.13 shows another setting you can enable, `ValidateOnBuild`, which goes one step further. When the setting is enabled, the DI container checks on application startup that it has dependencies registered for every service it needs to build. If it doesn't, it throws an exception and shuts down the app, as shown in figure 9.12, letting you know about the misconfiguration. This setting also has a performance cost, so it's enabled only in development environments by default, but it's useful for pointing out any missed service registrations.